

Defining sets

To define sets:

To define a set using **roster method**, explicitly list its elements. That is, start with $\{$ then list elements of the set separated by commas and close with $\}$.

To define a set using **set builder definition**, either form “The set of all x from the universe U such that x is ...” by writing

$$\{x \in U \mid \dots x \dots\}$$

or form “the collection of all outputs of some operation when the input ranges over the universe U ” by writing

$$\{\dots x \dots \mid x \in U\}$$

We use the symbol $\in$ as “is an element of” to indicate membership in a set.

Example sets: For each of the following, identify whether it's defined using the roster method or set builder notation and give an example element.

Can we infer the data type of the example element from the notation?

$$\{-1, 1\}$$

$$\{0, 0\}$$

$$\{-1, 0, 1\}$$

$$\{(x, x, x) \mid x \in \{-1, 0, 1\}\}$$

$$\{\}$$

$$\{x \in \mathbb{Z} \mid x \geq 0\}$$

$$\{x \in \mathbb{Z} \mid x > 0\}$$

$$\{\smile, \odot\}$$

$$\{\mathbf{A}, \mathbf{C}, \mathbf{U}, \mathbf{G}\}$$

$$\{\mathbf{AUG}, \mathbf{UAG}, \mathbf{UGA}, \mathbf{UAA}\}$$

Rna motivation

RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\mathbf{A}, \mathbf{C}, \mathbf{U}, \mathbf{G}\}$. Strands are ordered nonempty finite sequences of bases.

Formally, to define the set of all RNA strands, we need more than roster method or set builder descriptions.

Set recursive examples

Definition The set of nonnegative integers $\mathbb{N}$ is defined (recursively) by:

Basis Step:

Recursive Step:

Examples:

Definition The set of all integers $\mathbb{Z}$ is defined (recursively) by:

Basis Step:

Recursive Step:

Examples:

Definition The set of RNA strands S is defined (recursively) by:

Basis Step: $\mathbf{A} \in S, \mathbf{C} \in S, \mathbf{U} \in S, \mathbf{G} \in S$

Recursive Step: If $s \in S$ and $b \in B$, then $sb \in S$

where sb is string concatenation.

Examples:

Definition The set of bitstrings (strings of 0s and 1s) is defined (recursively) by:

Basis Step:

Recursive Step:

Notation: We call the set of bitstrings $\{0, 1\}^*$ and we say this is the set of all strings over $\{0, 1\}$.

Examples:

Defining functions more examples

Let's practice with functions related to some of our applications so far.

Recall: We model the collection of user ratings of the four movies Superman, Wicked, KPop Demon Hunters, A Minecraft Movie as the set $\{-1, 0, 1\}^4$. One function that compares pairs of ratings is

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \rightarrow \mathbb{R}$$

given by

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Notice: any ordered pair of ratings is an okay input to d_0 .

Notice: there are (at most)

$$(3 \cdot 3 \cdot 3 \cdot 3) \cdot (3 \cdot 3 \cdot 3 \cdot 3) = 3^8 = 6561$$

many pairs of ratings. There are therefore lots and lots of real numbers that are not the output of d_0 .

Recall: RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\mathbf{A}, \mathbf{C}, \mathbf{U}, \mathbf{G}\}$. The set of RNA strands S is defined (recursively) by:

$$\begin{array}{ll} \text{Basis Step:} & \mathbf{A} \in S, \mathbf{C} \in S, \mathbf{U} \in S, \mathbf{G} \in S \\ \text{Recursive Step:} & \text{If } s \in S \text{ and } b \in B, \text{ then } sb \in S \end{array}$$

where sb is string concatenation.

Pro-tip: informal definitions sometime use $\dots$ to indicate “continue the pattern”. Often, to make this pattern precise we use recursive definitions.

Name	Domain	Codomain	Rule	Example
<i>rnalen</i>	S	$\mathbb{Z}^+$	Basis Step: If $b \in B$ then $rnalen(b) = 1$ Recursive Step: If $s \in S$ and $b \in B$, then $rnalen(sb) = 1 + rnalen(s)$	$rnalen(\mathbf{AC}) \stackrel{\text{rec step}}{=} 1 + rnalen(\mathbf{A})$ $\stackrel{\text{basis step}}{=} 1 + 1 = 2$

<i>basecount</i>	$S \times B$	$\mathbb{N}$	Basis Step: If $b_1 \in B, b_2 \in B$ then $basecount((b_1, b_2)) =$ $\begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ Recursive Step: If $s \in S, b_1 \in B, b_2 \in B$ $basecount((sb_1, b_2)) =$ $\begin{cases} 1 + basecount((s, b_2)) & \text{when } b_1 = b_2 \\ basecount((s, b_2)) & \text{when } b_1 \neq b_2 \end{cases}$	$basecount((\mathbf{ACU}, \mathbf{C})) =$
------------------	--------------	--------------	--	---

“2 to the power of”	$\mathbb{N}$	$\mathbb{N}$	Basis Step: $2^0 = 1$ Recursive Step: If $n \in \mathbb{N}, 2^{n+1} =$	
---------------------	--------------	--------------	--	--

“ b to the power of i ”	$\mathbb{Z}^+ \times \mathbb{N}$	$\mathbb{N}$	Basis Step: $b^0 = 1$ Recursive Step: If $i \in \mathbb{N}, b^{i+1} = b \cdot b^i$	
-----------------------------	----------------------------------	--------------	--	--

$2^0 = 1$	$2^1 = 2$	$2^2 = 4$	$2^3 = 8$	$2^4 = 16$	$2^5 = 32$	$2^6 = 64$	$2^7 = 128$	$2^8 = 256$	$2^9 = 512$	$2^{10} = 1024$
-----------	-----------	-----------	-----------	------------	------------	------------	-------------	-------------	-------------	-----------------

Division algorithm

Integer division and remainders (aka The Division Algorithm) Let n be an integer and d a positive integer. There are unique integers q and r , with $0 \leq r < d$, such that $n = dq + r$. In this case, d is called the divisor, n is called the dividend, q is called the quotient, and r is called the remainder.

Because these numbers are guaranteed to exist, the following functions are well-defined:

div : $\mathbb{Z} \times \mathbb{Z}^+ \rightarrow \mathbb{Z}$ given by **div** ((n, d)) is the quotient when n is the dividend and d is the divisor.

mod : $\mathbb{Z} \times \mathbb{Z}^+ \rightarrow \mathbb{Z}$ given by **mod** ((n, d)) is the remainder when n is the dividend and d is the divisor.

Because these functions are so important, we sometimes use the notation $n \text{ div } d = \text{div} ((n, d))$ and $n \text{ mod } d = \text{mod} ((n, d))$.

Pro-tip: The functions **div** and **mod** are similar to (but not exactly the same as) the operators $/$ and $\%$ in Java and python.

Example calculations:

20 **div** 4

20 **mod** 4

20 **div** 3

20 **mod** 3

−20 **div** 3

−20 **mod** 3